COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 15, 2026	**DATE SUBMITTED:**	October 20, 2026
NAME:	Niña Denise M. Santiago Troy Jerron A. Bautista	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. __13.1__ **PARALLEL ALGORITHMS AND MULTITHREADING**

1. PROCEDURE OUTPUT

ILO A: Simple One-Threaded Example

Screenshot

```cpp
#include <iostream>
#include <string>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "================================" << std::endl;
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
    std::cout << "================================" << std::endl;
}

int main() {
    std::thread t1(print, 2026, "T.I.P. - QC");
    t1.join();
    return 0;
}
```

Output

```
================================
Printing integer: 2026
Printing string: T.I.P. - QC
================================
```

Analysis

In the program it creates a separate thread, **t1**, using std::thread to execute the print() function with the integer 2026 and the string "T.I.P. - QC" as arguments. Using this library allows the function to run independently from the main thread. The t1.join() makes the main thread wait until t1 completes the execution of its assigned task, ensuring that the program does not terminate immediately. Since the program performs a fixed number of

	printing operations regardless of the input size, its time complexity is (O(1)). Its space complexity is (O(1)) because it uses a fixed amount of additional memory for the thread and function arguments. Overall, this example clearly explains how threads execute tasks and how th.join() synchronizes the main thread with the completion of another thread.

Table 13-1. Simple One-Threaded Example

ILO B: Multi-Threaded Example	
Screenshot	

```cpp
#include <iostream>
#include <string>
#include <thread>
#include <vector>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P. - QC",
        "Competent",
        "Computer",
        "Engineers"
    };

    std::vector<std::thread> threads;

    for (int i = 0; i < static_cast<int>(s.size()); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (std::thread &th : threads) {
        th.join();
    }

    return 0;
}
```

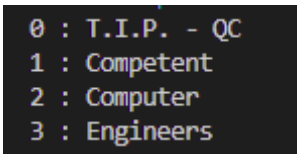
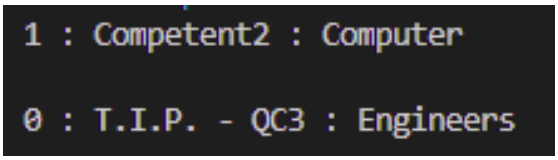
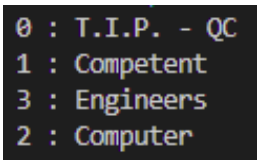
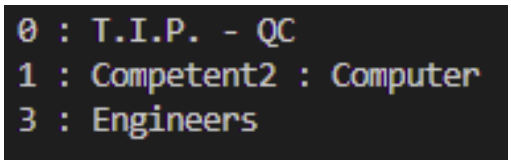
Output	   
Analysis	In this program unlike the first one, it creates multiple threads using a loop, assigning each thread a different string and index to process through the print() function. These threads can execute simultaneously (mentioned in the discussion), allowing multiple printing tasks to make progress without requiring the main thread to process each string sequentially. The th.join() makes the main thread wait for each specific thread to complete its execution, ensuring that all printing tasks are accomplished before the program ends. Since the program processes and prints (n) strings, its overall time complexity is (O(n)), however this is inaccurate as the actual execution time depends on thread creation overhead, CPU scheduling, and output operations according to our research. Its space complexity is (O(n)) because the vector stores (n) strings and another vector maintains (n) thread objects. The order of the output is not guaranteed because the operating system schedules the threads independently, even though every thread is joined before termination (shown in the execution output differences) . Overall, this example clearly explains how multithreading manages multiple tasks simultaneously, while the th.join() ensures proper completion and prevents the program from terminating while threads are still running.

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions.

Question 1 - Write a definition of multithreading and its advantages/disadvantages.

Santiago	*Bautista*
Multithreading is a specialized form of multitasking in which a program is divided into two or more threads that can execute concurrently. Each thread represents a separate path of execution within the same program, allowing multiple tasks to make progress while sharing the same memory space. **Advantages:** 1. **Improved performance:** Multithreading can speed up program execution by allowing multiple tasks to run simultaneously. 2. **Better responsiveness:** A program can continue handling other tasks while one thread is busy. 3. **Efficient resource utilization:** Threads share the same memory space, allowing efficient communication and sharing of resources. 4. **Scalability:** Multithreading allows programs to handle multiple tasks more effectively as available processing resources increase. **Disadvantages:** 1. **Complex programming and debugging:** Multithreaded programs are harder to develop and debug because threads may execute in unpredictable orders (Shown in the Procedure Part 2). 2. **Race conditions:** When multiple threads access and modify shared data without proper synchronization, they may produce incorrect	Multithreading is a technique in programming that divides a program into multiple threads that can run simultaneously. This allows your program to perform multiple operations, which are crucial in computing and programming, while still sharing the same memory space. **Advantages of Multithreading:** 1. **Improved performance** - Multiple tasks can run concurrently and utilize multi-core processing. 2. **Improved responsiveness** - Applications stay responsive while doing background tasks 3. **Ability to share resources** - Threads share the same memory space, making data exchange faster. 4. **Efficient distribution of tasks** - Allows performing independent tasks simultaneously. **Disadvantages of Multithreading:** 1. **Complex debugging** - Debugging multithreading can be difficult due to its behavior. 2. **Race Conditions** - Improper handling of resources may lead to race conditions or corrupted data 3. **Deadlocks -** Threads waiting on each other can cause deadlocks, which can halt their program execution. 4. **Overhead -** Creating and managing threads

or unexpected results. 3. **Deadlocks**: Threads may become stuck waiting for resources held by one another, preventing them from continuing execution. 4. **Additional overhead:** Creating, scheduling, and managing threads consumes processing time and resources, which can reduce performance for small tasks. References: - GeeksforGeeks. (2026, May 15). Multithreading in C++. https://www.geeksforgeeks.org/cpp/multithreading-in-cpp/ - TutorialsPoint. (n.d.). C++ multithreading. https://www.tutorialspoint.com/cplusplus/cpp_multithreading.htm	can add complexity **References:** GeeksforGeeks. (2026, August 31). Multithreading in C++. https://www.geeksforgeeks.org/cpp/multithreading-in-cpp/ CS Corner Sunita Ratai. (n.d.). Multithreading in C++. https://cscornersunitarai.com/courses/c/lessons/multithreading-in-c/?page_tab=overview

Question 2 - Rationalize the use of multithreading by providing at least 3 use-cases.	
Santiago	*Bautista*
Multithreading is useful because it allows multiple tasks to execute concurrently. Examples of use cases are: **1. Gaming:** Multithreading allows different tasks, such as physics calculations, artificial intelligence, audio processing, and networking, to run on separate threads. This helps games run more smoothly and improves responsiveness (Microsoft, 2017). **2. Web Servers:** Multithreading enables servers to handle multiple client requests concurrently instead of processing each request one at a time. This improves responsiveness and helps servers manage multiple users efficiently (Microsoft, n.d.). **3. Robotics:** Multithreading allows a robot to perform	Multithreading has many uses because of its ability to handle and execute tasks simultaneously. The common use of multithreading are: 1. **Web servers:** Multithreading allows web servers to handle multiple client requests. Each of the requests can be assigned to a different thread, which enables the server to process many requests without blocking other requests. 2. **Database Systems:** Modern database systems use multithreading to handle multiple queries and transactions simultaneously. This approach improves responsiveness, especially in high-concurrency areas.

several tasks concurrently, such as reading sensor data, processing camera images, controlling motors, and communicating with a control system. However, this has advantages that the builders and programmers need to take note of. **References:** - Microsoft. (n.d.). Multithreading with C and Win32. Microsoft Learn. https://learn.microsoft.com/en-us/cpp/parallel/multithreading-with-c-and-win32?view=msvc-170 - CppCon. (Feb. 02 2023). *Architecting multithreaded robotics applications in C++ - Arian Ajdari - CppCon 2022*. YouTube. https://www.youtube.com/watch?v=WzkI_FugMc0	3. **Image and Video Processing:** Applications dealing with processing images and videos can apply multithreading to perform different operations or different frames, significantly reducing processing time. **References:** Mildenberger, T. (2024, July 1). Introduction to Multithreading. Contabo. https://contabo.com/blog/introduction-to-multithreading/

Question 3 - Differentiate between parallelism and concurrency.	
Santiago	*Bautista*
Concurrency is the ability of a program to manage multiple tasks that make progress during overlapping periods. These tasks may take turns executing on a single CPU core, allowing the program to handle multiple operations efficiently. Just like the coffee machine example in today's discussion. Parallelism is the simultaneous execution of multiple tasks, typically using multiple CPU cores. It allows different tasks or portions of a problem to be processed at the same time, potentially improving performance. Just like the separation of the coffee machine to accommodate different customers.	Concurrency is when a program handles multiple tasks during the same period by taking turns working on them. Like a coffee machine serving different customers one after another. Parallelism is when multiple tasks are done at the exact same time. Like having several coffee machines serving different customers simultaneously.

References:

- GeeksforGeeks. (n.d.). *Difference between concurrency and parallelism*. https://www.geeksforgeeks.org/operating-systems/difference-between-concurrency-and-parallelism/
- ByteByteGo. (n.d.). *Concurrency is not parallelism*. https://bytebytego.com/guides/concurrency-is-not-parallelism/

Part B: Create C++ Code and show a solution that satisfies the given requirements below (Santiago)

Code

```cpp
#include <iostream>
#include <thread>
#include <mutex>
#include <string>

// Global variable
int globalVar = 0;
std::mutex mtx;

// Function that adds a value to the global variable
void add(int value) {
    std::lock_guard<std::mutex> lock(mtx);
    globalVar += value;
}

// Function to display the global variable
void display(const std::string& message) {
    std::lock_guard<std::mutex> lock(mtx);
    std::cout << message << globalVar << std::endl;
}

int main() {
    // Create three threads
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    // Display the current value
    display("Initial display: ");

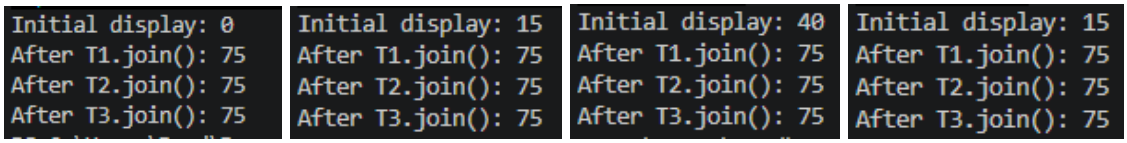
    // Wait for Thread 1
    T1.join();
    display("After T1.join(): ");

    // Wait for Thread 2
    T2.join();
    display("After T2.join(): ");

    // Wait for Thread 3
    T3.join();
    display("After T3.join(): ");

    return 0;
}
```

Output (Santiago)



4 tries of running the Code

Analysis (Santiago)

Based on the four trials, the program demonstrates how multithreading allows three threads to execute independently while sharing a single global variable. Based on the results, the global variable starts at 0, while T1 adds 15, T2 adds 25, and T3 adds 35. The expected final value is 75 because the three values are added to the same variable. However, the initial display shows different results in each trial, such as 0, 15, and 40. This happens because the threads begin executing immediately after they are created, so one or more threads may finish adding their values before the main thread displays the initial value. For example, an initial value of 15 means that T1 has already added its value, while an initial value of 40 may mean that T1 and T2 have already added their values.

The use of join() allows the main thread to wait for a specific thread to finish before proceeding to the next statement. However, calling T1.join() does not mean that only T1 can execute during that time. T2 and T3 may also finish their tasks while the main thread is waiting for T1. This explains why the displayed value after T1.join() is already 75 in all four trials. It is possible that all three threads have completed their additions before the main thread reaches the first display or finishes waiting for T1. The same value then appears after T2.join() and T3.join() because all the additions have already been completed.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result. (Santiago)

Header File Code



```
#ifndef BINARYSEARCH_H
#define BINARYSEARCH_H

#include <iostream>
#include <thread>
#include <mutex>
#include <vector>

std::mutex resultMutex;
int foundIndex = -1;

// Binary search function
void binarySearch(const std::vector<int>& arr,
                  int left, int right,
                  int target,
                  const char* threadName) {
    int low = left;
    int high = right;
    int result = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            result = mid;
            break;
        }
        else if (arr[mid] < target) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }

    {
        std::lock_guard<std::mutex> lock(resultMutex);

        if (result != -1 && foundIndex == -1) {
            foundIndex = result;
        }

        if (result != -1) {
            std::cout << threadName
                      << " found the target at index "
                      << result << ".\n";
        }
        else {
            std::cout << threadName
                      << " did not find the target in its section.\n";
        }
    }
}

#endif
```

Main File Code

```cpp

#include <iostream>
#include <vector>
#include <thread>
#include <algorithm>
#include <random>
#include <functional>
#include "binary_search.h"

int main() {
    int size;
    int target;

    std::cout << "=== Binary Search Using Multithreading ===\n";

    // Ask the user for the vector size
    std::cout << "Enter the number of elements: ";
    std::cin >> size;

    if (std::cin.fail() || size <= 0) {
        std::cout << "Invalid size. Please enter a positive integer.\n";
        return 1;
    }

    // Generate random numbers
    std::vector<int> arr;
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<int> dist(1, 100);

    for (int i = 0; i < size; i++) {
        arr.push_back(dist(gen));
    }

    // Sort the vector for binary search
    std::sort(arr.begin(), arr.end());

    // Display the generated vector
    std::cout << "\nGenerated sorted vector:\n";
    for (int num : arr) {
        std::cout << num << " ";
    }
    std::cout << "\n";
```

```cpp
    // Ask the user for the target
    std::cout << "\nEnter the number to search for: ";
    std::cin >> target;

    if (std::cin.fail()) {
        std::cout << "Invalid target. Please enter an integer.\n";
        return 1;
    }

    // Reset the shared result for this search
    foundIndex = -1;

    // Divide the vector into two sections
    int mid = size / 2;

    std::cout << "\nSearching using two threads...\n";

    std::thread T1(
        binarySearch,
        std::cref(arr),
        0, mid - 1,
        target,
        "Thread 1"
    );

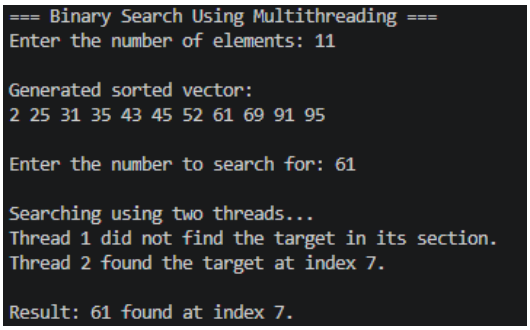
    std::thread T2(
        binarySearch,
        std::cref(arr),
        mid, size - 1,
        target,
        "Thread 2"
    );

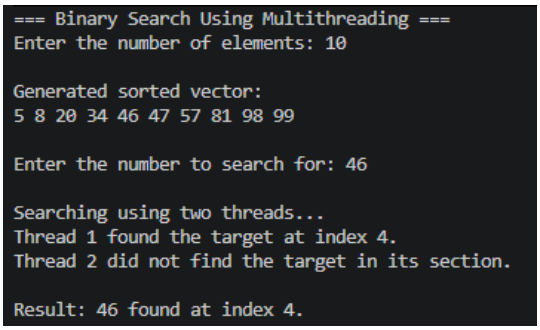
    // Wait for both threads
    T1.join();
    T2.join();

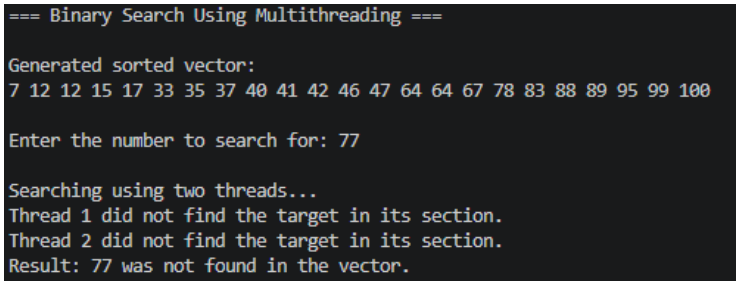
    // Display the final result
    if (foundIndex != -1) {
        std::cout << "\nResult: " << target
                  << " found at index "
                  << foundIndex << ".\n";
    }
    else {
        std::cout << "\nResult: " << target
                  << " was not found in the vector.\n";
    }

    return 0;
}
```

Output (Santiago)







Analysis (Santiago)

Based on the output, the program demonstrates how multithreading can be applied to binary search by dividing a randomly generated vector into two sections and assigning each section to a separate thread. The program first generates random integers and sorts them in ascending order because binary search requires sorted data to work correctly. The user then enters a target number to search for. In the first trial, the target value 61 was found by Thread 2 at index 7, while in the second trial, the target value 46 was found by Thread 1 at index 4. In the third trial, the target value 77 was not found because it was not present in the generated vector. These results show that the program can successfully identify a target value when it exists and report when it cannot be found.

The results also demonstrate that the two threads can execute concurrently, so the order of their output may vary between runs. Each thread searches only its assigned section, and the join() function ensures that both threads finish before the program displays the final result. The use of random numbers also means that the vector and search results can change every time the program runs. However, multithreading does not automatically make binary search faster because binary search already reduces the search area by half during each step. Dividing the vector into two sections and creating separate threads also introduces additional processing overhead. In conclusion, this part of the activity demonstrates the use of binary search, random data generation, and multithreading in one program. It also shows how threads can work on separate parts of the same dataset to search for a target value, although the actual performance advantage depends on the size of the vector and the system running the program.

Conclusion / Analysis (Santiago)
In this activity, we learned how concurrency, parallelism, and multithreading work in programming. We learned how to create threads, assign tasks to them, and use join() to make sure they finish before the program ends. We also observed that threads may execute in different orders, which can affect the output, especially when they share the same variable. Through part C of the activity (Binary Search w/ Multithreading), we learned how to generate random numbers, sort a vector, and divide the search into two sections handled by separate threads. Overall, this activity helped us understand how multithreading can manage tasks more efficiently, while also showing the importance of synchronization to prevent errors. We also learned that multithreading does not always make a program faster, especially when the task is simple or the data is small.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ***Grammarly***

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: ***Grammar correction***

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Grammarly was used to correct grammar corrections and correct vocabulary usage; it did not replace original thinking, analysis, authorship, or completion of the required work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.